



NetSentinel

From a notebook to a live, human-approved detection platform

What was built, in four numbers

19/19

end-to-end checks
passed on the live,
hardened system

100%

recall on never-seen
brute-force attacks
(48% before v2)

12

pipeline steps, from
ingest to dashboard,
running as 9 services

145

automated tests,
including security
and tamper tests

Machine learning decides; Claude explains; a human approves every action.

The starting point

A notebook whose results looked strong but could not be trusted

Five problems behind 96% accuracy



Target leakage

'Attempted Category' was a model input and marks exactly the '... - Attempted' labels



Not a time split

Files loaded alphabetically, so the 'future' test set held attack types absent from training



Identity, not behaviour

Source and destination IPs ranked #4 and #7: the model memorised the lab's addresses



Wrong ROC-AUC

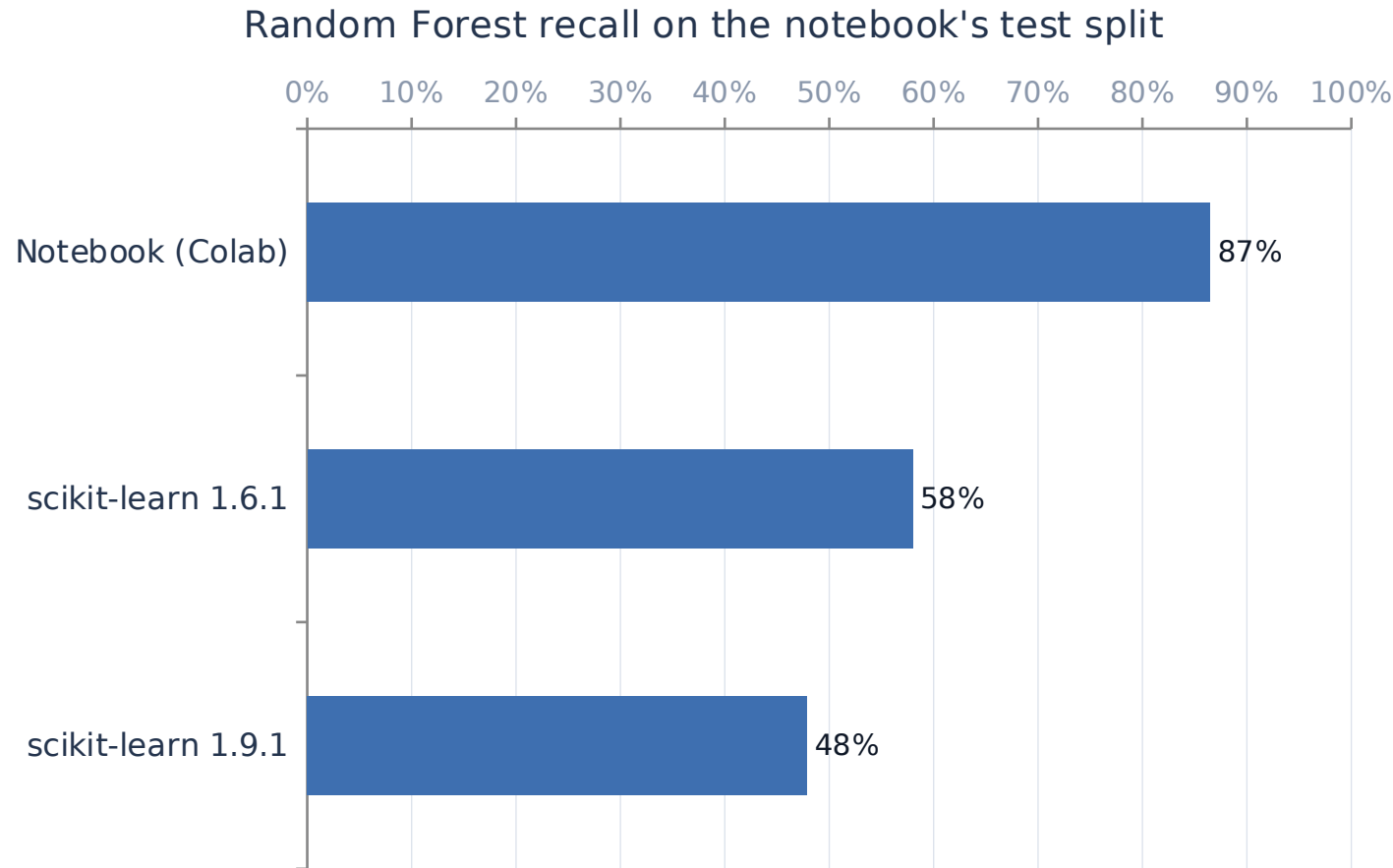
Computed from 0/1 predictions for two models: balanced accuracy reported as AUC



Binary only

Attack / not attack, with no family or type for triage

Same code and seed, three different answers



The data pipeline reproduced exactly

3,167 duplicates, 27 dropped features, 1,467,762 / 629,042 rows: identical.

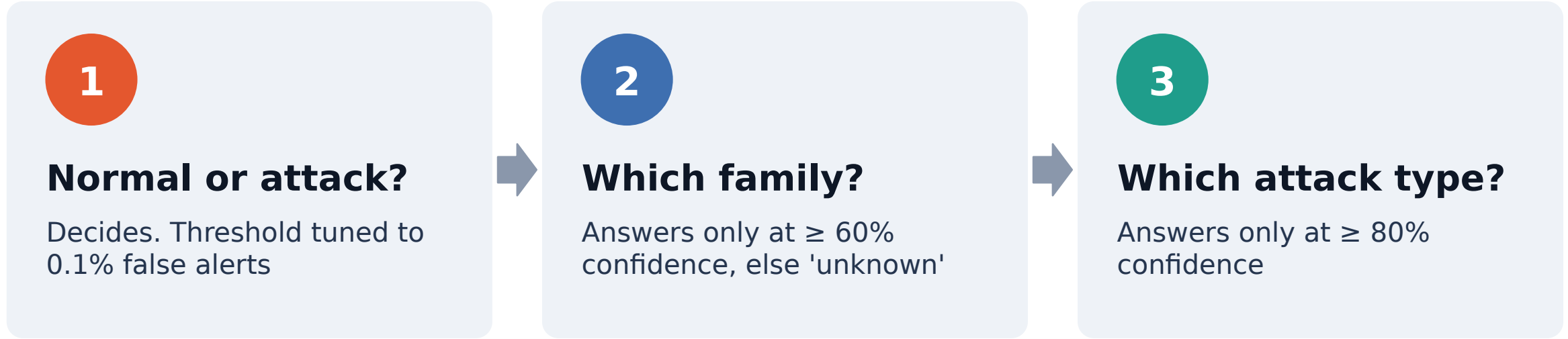
The headline model did not

Its test attacks were never seen in training, so catching them depended on incidental tree splits.

The detection model

v1: a clean feature set and a hierarchy · v2: each source's last 60 seconds

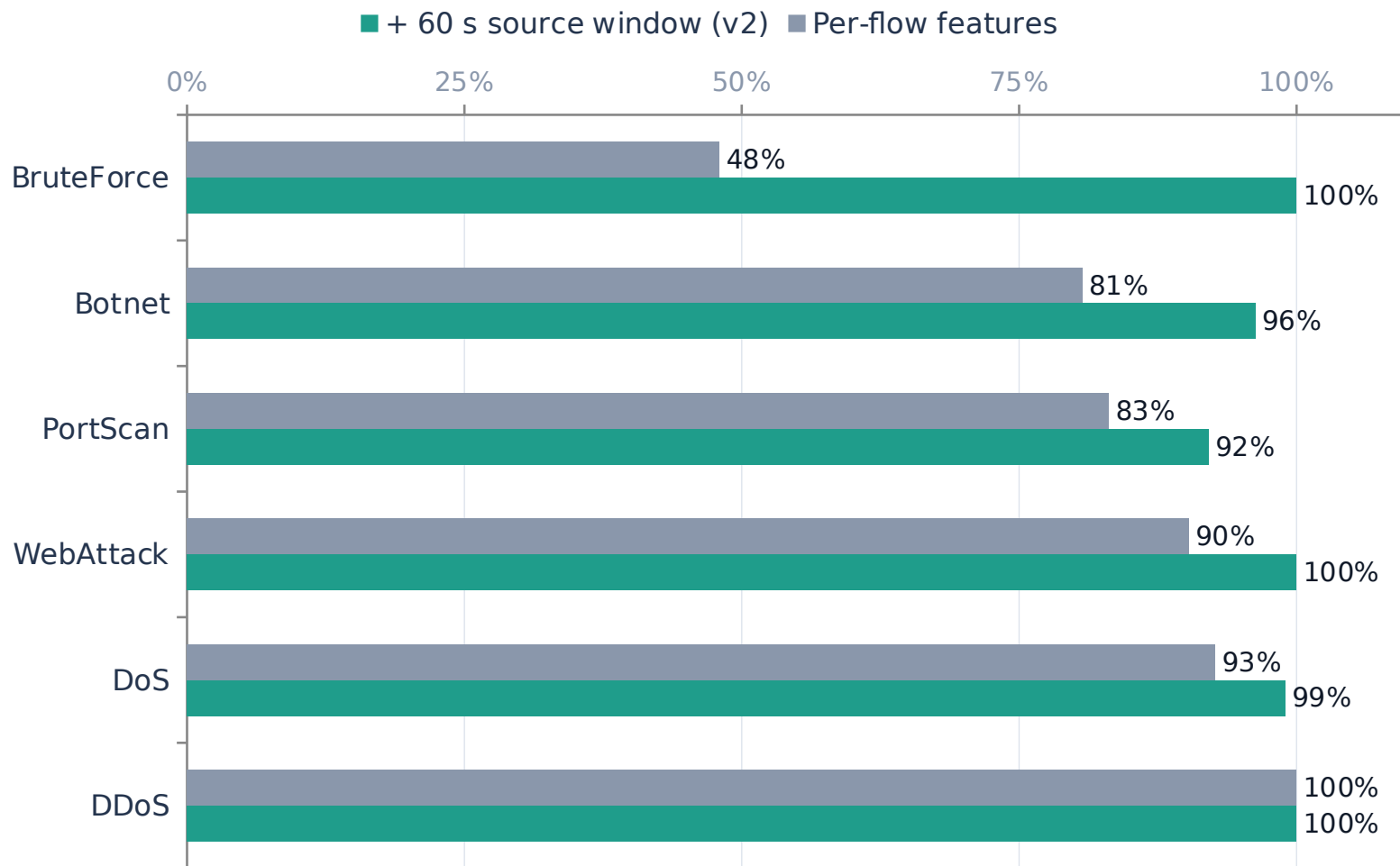
Three questions, and only the first one decides



Clean inputs: 83 fixed features (no leakage, no IPs or ports-as-identity) · missing values kept, not dropped · real duplicates removed (259k, not 3k)

Attack types seen in training: 99.999% recall at 0.09% false alerts. Treat as a ceiling: CICIDS-2017 is easy when tools repeat.

Window features close the unseen-attack gaps



How it was tested

Each family is removed from training, then tested at the same 0.1% false-alert budget.

Why it works

A scan probe looks like a handshake; 997 ports in a minute does not. Features are causal: only flows already ended count.

v2 caught every attack in real replayed traffic

Real traffic slice	Attacks caught	Family correct	False alarms
Port scan (Fri, 1 min)	997 / 997	100%	0 / 279
FTP brute force (Tue, 2 min)	124 / 124	100%	2 / 1,816
SSH brute force (Tue, 2 min)	102 / 102	100%	6 / 1,318
Botnet attempts (Fri, 3 min)	42 / 42	100%	6 / 1,645



Every attack flow caught and named

False alarms ran higher than the 0.10% seen in validation; all scored just above v2's very low threshold.

1,265 / 1,265 **0.28%**

attack flows caught

false alarms on 5,058 benign flows

The platform

Twelve steps, six agents, one human in control

From a network flow to a ticket in one pipeline



Ingest

API key, validation



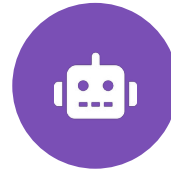
Detect

Model Check, v2
model



Correlate

cases by source



Investigate

Claude, guarded



Score risk

explainable rules



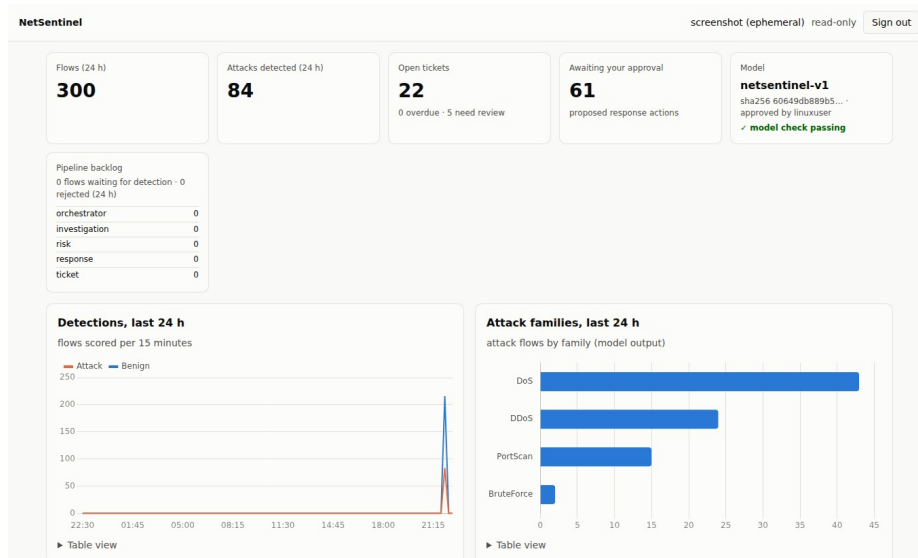
Propose

reversible only



Ticket

SLA, timeline



Every investigation keeps four labelled sections

VERIFIED TELEMETRY · MODEL OUTPUT · LLM ANALYSIS · RECOMMENDATION

Claude cannot change a detection or a risk score. A guard rejects any analysis naming an IP that is not in the evidence, and strips containment steps from its advice.

A human approves; secrets stay out of code



Human approval

Response actions are a closed list of reversible steps. The database refuses any decision by an agent; dashboard decisions re-ask for your key.



Secrets in OpenBao

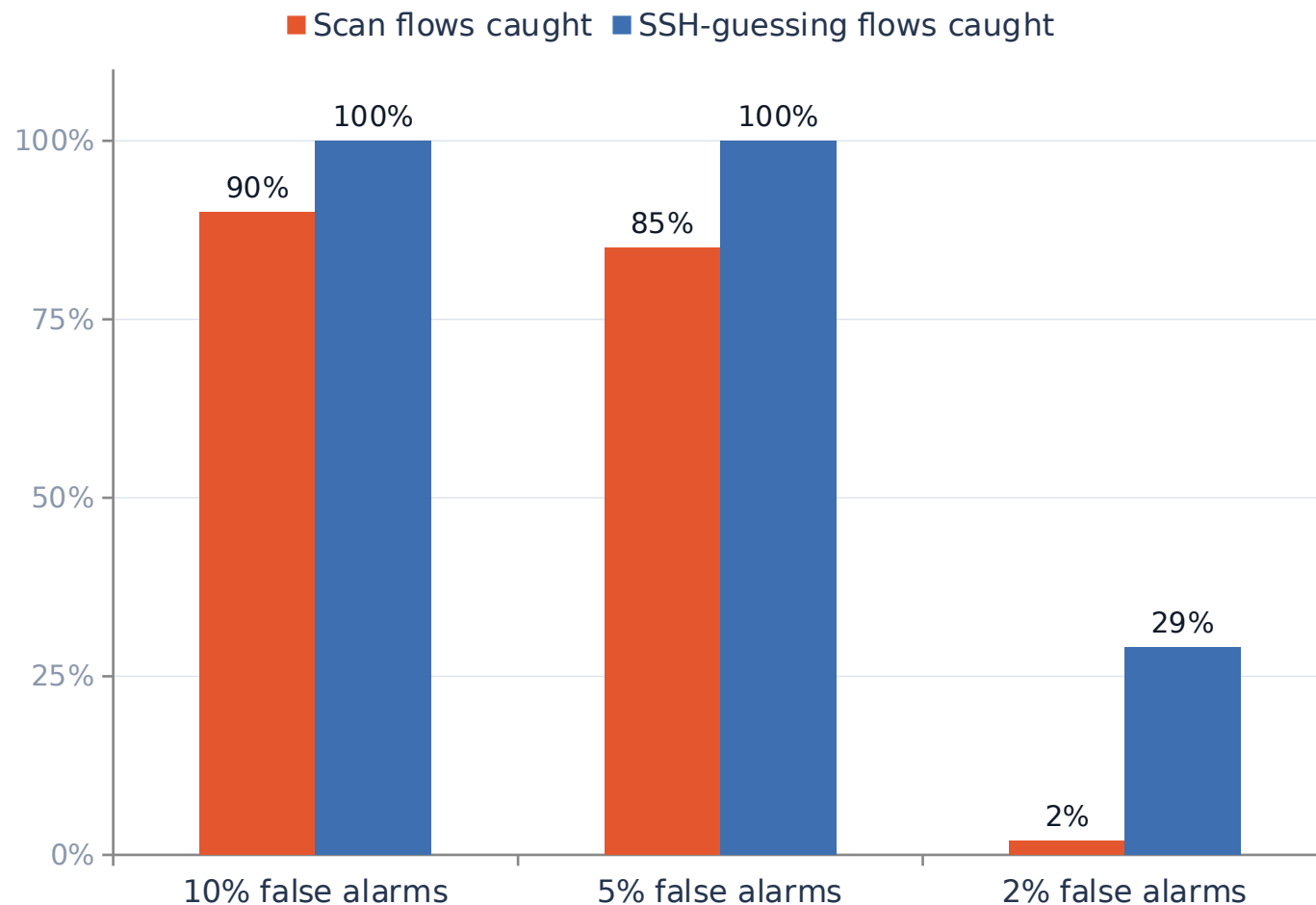
Database password rotates daily through the Security Wall. API keys are stored only as hashes; you created yours on your own PC.



Hardened services

Sandbox score 8.5 → 1.6 (systemd). HTTPS with Let's Encrypt, dashboard limited to your IPs, model files hash-checked before loading.

Live traffic: ranking holds, thresholds don't



Labels from the server's own logs

Firewall blocks mark scanners; failed SSH logins mark password guessing. ROC-AUC 0.95.

Decision: alerts stay off

The sensor runs in shadow mode (scored, no tickets) until calibrated on a day of data.

Next steps

- Calibrate on 24 h of live traffic (netsentinel calibrate)
- Retrain on features from the same flow meter as the sensor
- Approve a per-source alert threshold, then switch the sensor to --alert